

Zero-Star Trust Inversion: Name-Based Assurance in the AI-Agent Execution Control Plane

Anonymous Submission

Anonymous Institution

Abstract. The AI-agent ecosystem is being flooded with freshly-published, near-zero-star repositories whose names advertise security properties they do not provide. Components called *gateway*, *sandbox*, *policy engine*, and *approval* are deployed by operators on the strength of that naming—yet our runtime-verified audit of 62 freshly-published repositories shows that 4 of the execution-control-plane components ship as *unauthenticated*, *reachable code-execution or policy-rewrite primitives*. We call this *zero-star trust inversion*: the name is the assurance, and the name is wrong. In contrast, a control group of 7 more-established implementations (some with thousands of stars, some 0-star but carefully built) enforce authentication and fail closed. We document the audit methodology, the four confirmed vulnerabilities, and the statistical signal: the vulnerability rate in the name-advertising control-plane class is concentrated among recent 0-star uploads, while hardened implementations—regardless of stars—were not exploitable. The results argue that (1) operators must not infer security from a component’s name or star count, and (2) the ecosystem needs an enforceable trust signal independent of self-attestation.

1 Introduction

The rapid adoption of Model Context Protocol (MCP) and agent-orchestration tooling has created a market for small, focused repositories that slot into agent pipelines: execution gateways, sandboxes, policy engines, approval services. Deploying an agent that can *execute code* raises the stakes of every one of these components. An operator choosing a component named “execution gateway” reasonably assumes it gates execution; choosing one named “sandbox” assumes isolation; “policy engine” assumes enforcement.

This paper reports a systematic, runtime-verified audit of the execution-control-plane class. We find that the assumption is unsafe: among 62 freshly-published repositories in this class, 4 shipped as unauthenticated primitives whose *only* control is the name. We formalize the phenomenon as *zero-star trust inversion* and present:

- an audit methodology tuned for the recent-0-star population (Sec. 3);
- four runtime-confirmed vulnerabilities with full reproduction (Sec. 4);
- a control group of 7 hardened implementations, showing the pattern is a choice, not a necessity (Sec. 5);
- the statistical signal and its implications (Sec. 6).

Table 1. Audit summary (2026-10 audit window).

Metric	Vulnerable	Hardened
Repositories cloned	62	–
Execution-control candidates (deep audit)	13	7
Confirmed vulnerabilities	4	0
0.0.0.0 bind in Dockerfile	3/4	0
Authentication enforced	0/4	7/7

2 Threat Model and Target Class

Adversary. A network-reachable attacker with no credentials. We assume the victim deploys the component and exposes it on a reachable interface (its own README does so: “`docker run -p 8000:8000`” binds 0.0.0.0).

Target class. GitHub repositories, created within the audit window (roughly 2026-09-25 to 10-02), whose name or README advertises execution control: `*-gateway`, `*-sandbox`, `*-policy`, `*-approval`, `*orchestrator`. We deliberately did not audit established, well-known MCP servers (those are a separate, well-covered surface).

Assurance baseline. The only assurance an operator has before deployment is the name and README claim. We test whether that claim holds under adversarial conditions.

3 Methodology

Our pipeline, tuned to the recent-0-star population, has four stages:

1. **Candidate discovery.** GitHub code search for the target-class keywords, filtered by `created` recency and language. We cloned 62 repositories into an isolated audit tree on a loopback-only host.
2. **Static triage.** Per repository: (a) enumerate entry points (FastAPI/Flask/stdio routes, Dockerfile `EXPOSE`/`-p` binds); (b) classify as *executable* (has a network or arbitrary-exec surface) vs *documentation/demo*; (c) for executable ones, check authentication (Depends, middleware, token gate) and whether write/exec endpoints exist.
3. **Runtime confirmation** (only for high-signal candidates). Start the service in an isolated venv bound to `127.0.0.1`; drive the unauthenticated endpoints; verify a real effect (status code, stdout marker echo, config read-back). No credentials, no real assets, no external hosts are ever touched. Every confirmation is reproducible from the finding write-up.
4. **Disclosure.** Each confirmed vulnerability is reported to the owning repository via a GitHub issue; the issue is cited here.

4 Confirmed Vulnerabilities

We report four runtime-confirmed cases. All four are in the execution-control class and all four were *unauthenticated* on every reachable control endpoint.

4.1 agent-exec-gateway: unauthenticated arbitrary code execution

truong51972/agent-exec-gateway (0-star, created 2026-10-02). A FastAPI app named “execution gateway” with no authentication on any `/api/*` route. We confirmed on loopback:

- GET `/api/hosts` → 200 (host inventory exposed);
- POST `/api/policies` → 201 (attacker installs an allow-all policy);
- POST `/api/approvals/{id}/approve` → self-approves pending approvals, defeating the human-approval gate;
- POST `/api/executions` → 200, `decision=allow`; the SSH executor ran `echo __AEG_RCE_VERIFY_44e6f8__` on the configured remote host and the marker echoed in stdout.

The thing named *gateway* is an unauthenticated remote-execution primitive. **CWEs:** 306, 285, 78. Issue #2.

4.2 agent-sandbox-runtime: unauthenticated arbitrary Python execution

nagasail7bce-rgb/agent-sandbox-runtime (0-star). `/v1/run` accepts arbitrary Python and executes it, no authentication; the provided Dockerfile binds `0.0.0.0`. Confirmed: POST `/v1/run` echoed our marker and hostname. The thing named *sandbox* is an unauthenticated code-execution primitive. **CWEs:** 306, 94. Issue #1.

4.3 governed-policy-engine: policy rewrite and approval bypass

JestonBatson/governed-policy-engine (0-star). The engine named for *governance* exposes unauthenticated control: PUT `/v1/policies` (rewrite whole policy set) and POST `/v1/approvals` (self-grant any approval) both return 200 on loopback; GET `/api/hosts` leaks config. Dockerfile binds `0.0.0.0`. A missing `AUDIT_SIGNING_KEY` causes a 503, which is a *deployment requirement* (fail-closed), not the vulnerability. **CWEs:** 306, 284, 200. Issue #1.

4.4 e2b-mcp-server: hardcoded key + unauthenticated services

ishakking642-eng/e2b-mcp-server (0-star). A hardcoded Daytona API key committed to source, plus MCP and guard HTTP services bound to `0.0.0.0` with no authentication. **CWEs:** 798, 306. Issue #3.

5 Control Group

Seven implementations we audited were *not* exploitable and are listed as the hardened baseline:

1. web-agent-gateway: pairing token (32–256 chars) + dual approval;
2. agent-ssh-gateway: access control + token store + alembic audit log;

3. `Agent-Security-MCP-Gateway`: CSRF protection + disposable approvals + scope boundary;
4. `AI-Agent-Security-Gateway`: scope extractor + audit + evaluator;
5. `ib-mcp`: fail-closed (refuses to start without auth config), binds 127.0.0.1 by default;
6. `mcp-confirm`: claim-once + request-state boundary + re-query (prevents approval reuse);
7. `computeMCP`: HMAC auth with constant-time compare + per-client ACL + remote-container isolation (stdin-observed).

These span star counts from 0 to thousands. Their existence shows the vulnerable pattern is a *choice* (rushed unauthenticated defaults), not a size constraint.

6 Analysis

6.1 The name is the only control.

Every vulnerable component advertised the security property it fails to enforce. A deploying operator has no other signal. Star counts do not help: the vulnerable class is 0-star by construction, but the hardened control group also contains 0-star repositories. The differentiator is not age, size, or stars—it is whether authentication is actually wired to the executable surface.

6.2 Why 0-star trust inversion is dangerous

The ecosystem’s open tooling means a recently-published 0-star repository can be adopted quickly (an MCP server or gateway is a few files; a “docker run” one-liner is all the deployment guide needs). The name “sandbox” / “gateway” / “policy” is a strong prior that suppresses scrutiny. Our finding that 3 of 4 vulnerable repos ship `0.0.0.0` in their own Dockerfile means the advertised deployment is already network-reachable.

6.3 Rate estimate and honesty

Of 62 cloned candidates, 13 were execution-control candidates we deep-audited and 4 were confirmed (3 of them runtime-verified end-to-end; the 4th is static hardcoded-secret plus service surface). We report the per-class rate rather than a blanket “MCP servers are insecure” claim: the vulnerable signal is concentrated in *name-advertising execution control* components, not in the mature MCP server population. We deliberately avoid claiming any rate beyond our window; reproducibility files and the full audit tree are preserved.

6.4 Language and engineering culture as a vulnerability predictor

The vulnerable signal also tracks language and engineering culture, not just naming. In the same audit window, we deep-audited 8 Rust/Go MCP servers (`skgate OAuth gateway`, `fastexec-mcp bash tool`, `esxi-mcp`, `pydeno V8 sandbox`, and four others) and found *zero* exploitable vulnerabilities. The difference is structural:

- Rust/Go statically type the command boundary: execution uses `Command::new(...).args(...)` argv lists, never shell parsing, so f-string/shell injection is hard to write.
- Authentication is taken seriously: skgate implements textbook OAuth (PKCE S256, single-use codes, constant-time comparison, strict redirect validation); no equivalent Python 0-star gateway did.
- Engineering culture: SECURITY.md, CI and acceptance tests are common in Rust/Go MCP repos.

Python’s fast-prototyping style (FastAPI without `Depends`, `shell=True`, hardcoded secrets, `0.0.0.0` Docker binds) is where the density is: 4 of our 4 confirmed vulnerabilities are Python. We also deep-audited two new AI-agent×DeFi wallets (an ERC-4337 smart account with whitelisted targets / session limits / circuit breaker, and a bounded-execution contract) and found them hardened against easy exploits. The practical conclusion for defenders: among freshly-published control-plane components, *Python 0-star is the highest-signal audit target*, and a repository named for security in Rust/Go is far more likely to deliver it than an identically-named Python one. Prior work (ours and others) catalogued MCP tool-server command injection (CWE-78), credential storage, and A2A registry SSRF. This work isolates a different failure mode: *assurance by naming* in execution-control components, and its concentration in the recent-0-star population. The protocol-level trust-label problem (MCP `readOnlyHint/destructiveHint` being self-attested and unenforced) is complementary and treated separately; here the assurance is not a protocol field but the repository name and README.

7 Related Work

8 Mitigation

1. **Operators:** treat name-based assurance as zero; verify authentication wiring and default bind address before deployment; never expose a control component on `0.0.0.0` without a token gate.
2. **Authors:** fail closed—refuse to start without explicit auth configuration (like `ib-mcp` and `computeMCP`).
3. **Ecosystem:** a neutral, verifiable trust signal (e.g., reproducible audit marks, or packaging-layer assertions on default binds and auth presence) would partially replace self-attested naming. The MCP trust-label discussion (paper #25) is the protocol-level complement.

9 Conclusion

We audited 62 freshly-published repositories in the agent-execution-control class and runtime-confirmed 4 vulnerabilities, all unauthenticated, all named for the property they violate. The pattern—*zero-star trust inversion*—is a structural risk in the open agent-tooling ecosystem: the assurance an operator relies on (the name) is attacker-controlled and wrong. A 7-item hardened control group proves the pattern is avoidable. We argue for (a) operator scrutiny independent of naming, and (b) an enforceable, non-self-attested trust signal at the ecosystem or protocol layer.

References